

AgentHub IM 与 Clowder 技术文档

版本: v1.1 日期: 2026-06-10 关联 PRD: [prd产品需求文档.md](#) 覆盖范围: [sections/im](#)、[sections/clowder-ai](#)、[sections/ui](#)

1. 总览

AgentHub IM 与 Clowder 的集成是三个独立子系统之间的协作，各自有明确的边界：

- [sections/ui](#) 负责用户可见的 IM 工作台、消息状态、Agent 目录、技能、项目群、部署卡和跨端布局。
- [sections/im](#) 负责 TangSengDaoDao 业务 API、WuKongIM 通讯、消息持久化、群组/好友/文件能力，以及 Clowder bridge。
- [sections/clowder-ai](#) 负责 Agent 团队本体：thread、connector、命令、路由、技能、记忆、协作记录、出站投递和运行时调用。

硬约束：浏览器不碰 Clowder 密钥，IM 后端不复制 Clowder 路由逻辑，Clowder 不替代 TangSeng/WuKongIM 的 IM 存储。

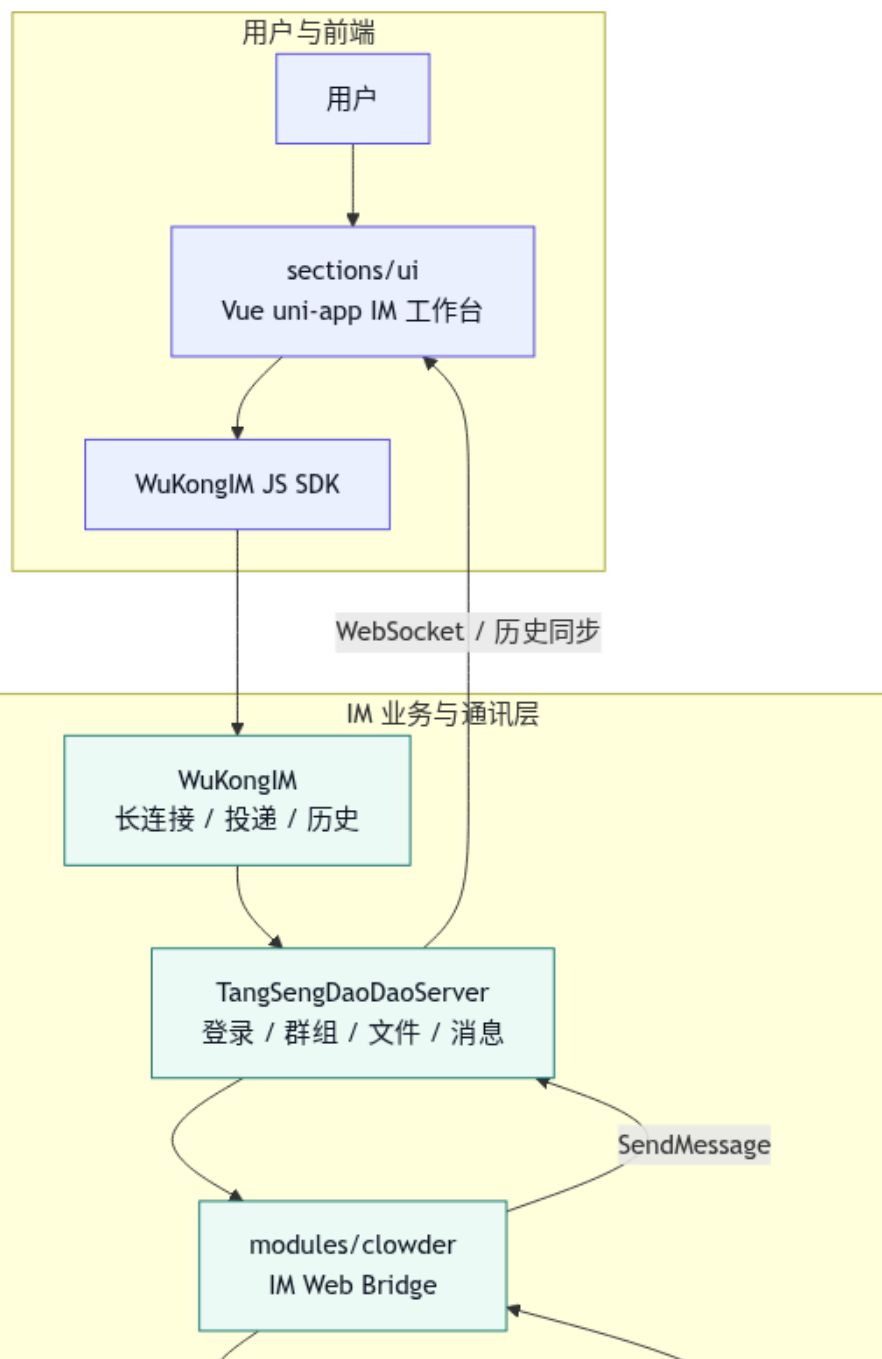
1.1 按角色阅读

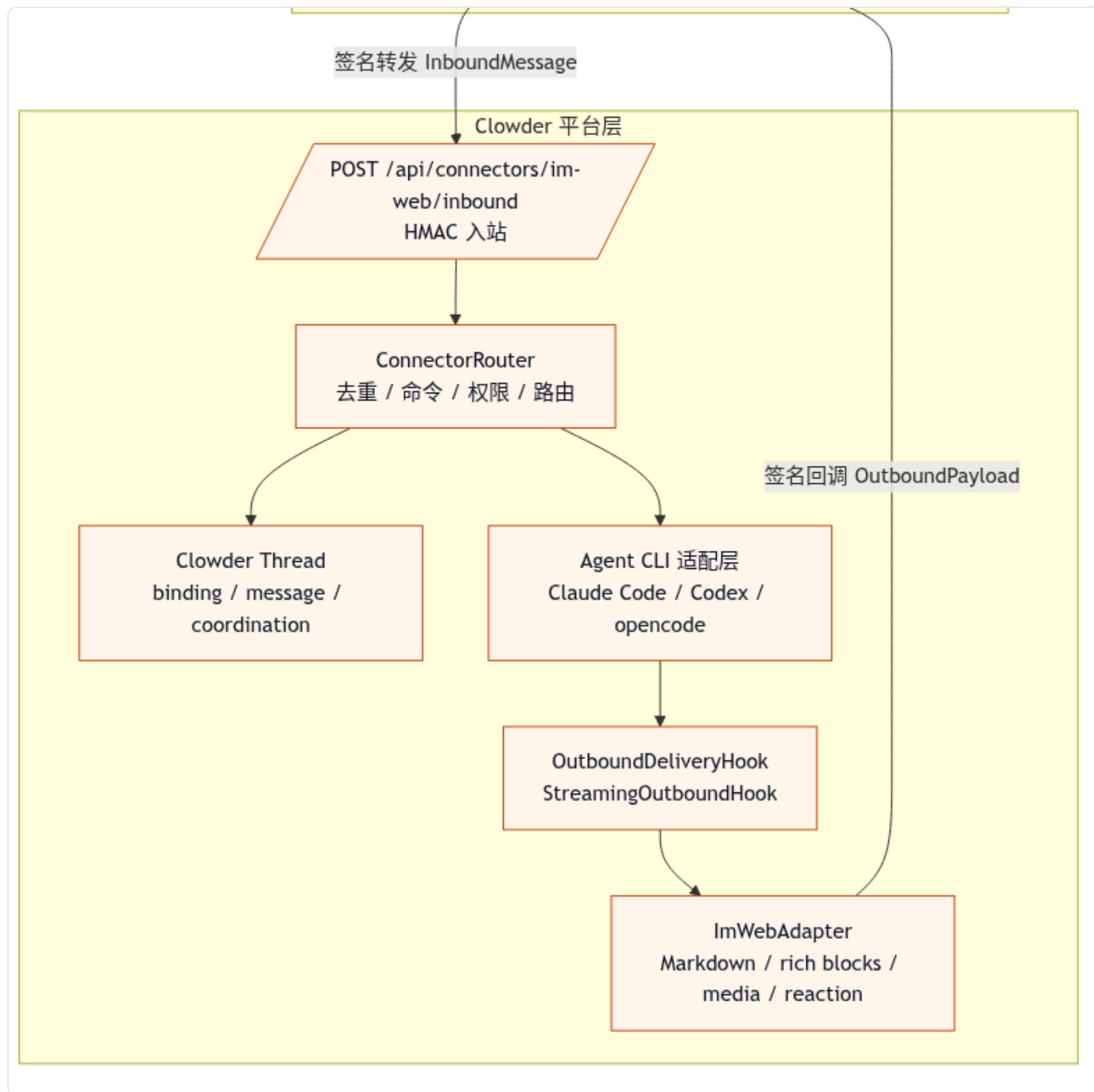
角色	优先阅读	可以跳过
产品 / 评审	§ 1 总览、§ 2 架构图、§ 7 消息生命周期、§ 15 测试与验证	§ 6 数据结构、§ 9 鉴权细节、§ 12-13 后端/路由要点
前端开发	§ 1 总览、§ 3 源码地图（前端部分）、§ 6.3 UI 消息对象、§ 7 消息生命周期、§ 8.1 接口、§ 11 前端实现要点	§ 12-13 后端/路由要点
后端开发	§ 1 总览、§ 3 源码地图（后端部分）、§ 6.1-6.2 数据结构、§ 8 接口、§ 9 鉴权、§ 10 幂等恢复、§ 12-13 后端/路由要点	§ 11 前端实现要点
测试	§ 1 总览、§ 7 消息生命周期、§ 8 接口、§ 10 幂等恢复、§ 15 测试与验证	§ 11-13 实现要点

2. 架构图

图源文件（[.mmd](#) 为 Mermaid 源文件，[.md](#) 为渲染中间格式，[.png](#) 为导出静态图）：

- [./figures/agenthub-im-clowder-architecture.mmd](#)
- [./figures/agenthub-im-clowder-architecture.md](#)
- [./figures/agenthub-im-clowder-architecture.png](#)





3. 源码地图

区域	主要文件	职责
前端 API 聚合	<code>sections/ui/services/native-im/service.js</code>	登录、SDK 初始化、消息同步、Clowder API、技能、项目群、部署请求
前端消息状态	<code>sections/ui/services/native-im/message-state.js</code>	消息摘要、去重、流式合并、prompt context、reaction、pending feedback
前端 Agent 状态	<code>sections/ui/services/native-im/agent-state.js</code> 、 <code>sections/ui/stores/agent.js</code>	Clowder Agent 目录、直聊联系人、技能、角色模板、删除屏蔽

区域	主要文件	职责
前端消息 Store	sections/ui/stores/message.js	本地乐观插入、发送、同步、项目群卡、部署卡、模板 Agent 卡
前端聊天 UI	sections/ui/components/chat/*.vue	消息气泡、输入区、文件卡、Clowder 面板、右侧工作区
TangSeng bridge 配置	sections/im/TangSengDaoDaoServer/modules/common/clowder_config.go	IM_WEB_CLOWDER_ENABLED、Clowder API、secret、owner、超时
TangSeng 签名	sections/im/TangSengDaoDaoServer/modules/common/clowder_signature.go	HMAC SHA-256 签名与时间窗校验
TangSeng Clowder 模块	sections/im/TangSengDaoDaoServer/modules/clowder/*.go	API、inbound/outbound 归一化、项目群、技能、manual context pins、媒体
IM 消息监听	sections/im/TangSengDaoDaoServer/modules/message/event.go	注册消息监听器，把 IM 消息转发到 Clowder
Clowder connector 类型	sections/clowder-ai/packages/shared/src/types/connector.ts	im-web connector definition、thread binding 类型
Clowder inbound	sections/clowder-ai/packages/api/src/infrastructure/connectors/ImWebInboundHandler.ts	校验 IM 签名，调用 ConnectorRouter.route()
Clowder 路由	sections/clowder-ai/packages/api/src/infrastructure/connectors/ConnectorRouter.ts	去重、白名单、命令、thread binding、@ 解析、coordination 记录
Clowder 命令	sections/clowder-ai/packages/api/src/infrastructure/connectors/ConnectorCommandLayer.ts	/where、/new、/use、/threads、/focus、/ask、/allow-group 等
Clowder 出站	sections/clowder-ai/packages/api/src/infrastructure/connectors/adapters/ImWebAdapter.ts	发送 Markdown、rich blocks、媒体、reaction、流式 placeholder/chunk/final
Clowder 投递 hook	OutboundDeliveryHook.ts、StreamingOutboundHook.ts	thread 绑定查找、文件发布、语音 fallback、流式清理
角色模板	sections/clowder-ai/cat-template.json	模板 Agent 的名称、别名、简介、能力和默认模型

4. 运行时拓扑

4.1 浏览器 / uni-app

前端通过 nativeImService 统一访问 TangSeng API。普通文本和媒体优先走 WuKongIM JS SDK；Clowder 直聊和结构化能力走 /v1/clowder/*。

主要状态由 Pinia 管理：

Store	状态
app.js	登录用户、主题、通知、全局运行态
navigation.js	当前模块、桌面/移动导航
conversation.js	会话列表、选中会话、草稿、未读
message.js	消息列表、发送状态、卡片状态、同步状态
agent.js	Agent 列表、技能、OAuth 能力、模板
group.js	群信息、成员、群智能体
file.js	文件列表和预览
settings.js	设备、主题、通知和隐私设置

4.2 TangSengDaoDao / WuKongIM

TangSengDaoDao 是 IM 业务层。它处理用户、好友、群、文件、消息 API 和 Clowder bridge。WuKongIM 是通讯层，负责长连接、消息投递和消息存储。

Clowder bridge 作为 TangSeng 模块注册：

- `modules/clowder/1module.go` 注册 API 和 SQL。
- `modules/message/event.go` 在 bridge 配置完整时注册消息监听器。
- `modules/clowder/api.go` 暴露 `/v1/clowder/*` 和 `/api/im-web/clowder/outbound`。

4.3 Clowder

Clowder 把 `im-web` 当作一个 connector。它收到 inbound 后，使用 `ConnectorRouter` 完成以下动作：

- 去重。
- 群白名单和命令权限检查。
- 处理 slash command。
- 查找或创建 thread binding。
- 存储 connector message。
- 解析 @ 和显式目标 Agent。
- 触发 Agent invocation。
- 通过 `OutboundDeliveryHook` 和 `ImWebAdapter` 回写 IM。

5. 配置

TangSeng bridge 从环境变量读取配置：

环境变量	说明	默认
IM_WEB_CLOWDER_ENABLED	是否启用 Clowder bridge	false
CLOWDER_API_BASE_URL	Clowder API 地址	空
CLOWDER_CONNECTOR_ID	connector ID	im-web
CLOWDER_CONNECTOR_SECRET	双向 HMAC secret	空
CLOWDER_DEFAULT_OWNER_USER_ID	TangSeng 代理访问 Clowder 时使用的 owner	空
CLOWDER_REQUEST_TIMEOUT_MS	请求 Clowder 超时	5000
CLOWDER_SIGNATURE_TOLERANCE_MS	签名时间窗	300000

前端可配置：

配置	来源	默认
TangSeng API base URL	native_api_base_url storage、VITE_TANGSENG_API_BASE_URL 、 VITE_API_BASE_URL	/v1/
Clowder direct fallback base URL	clowder_api_base_url storage、 VITE_CLOWDER_API_BASE_URL	/clowder-api/api/

6. 数据结构

6.1 InboundMessage

TangSeng 将 IM 消息归一为 InboundMessage 后发给 Clowder：

字段	含义
connectorId	固定为 im-web
externalChatId	"{channelType}:{channelIdOrFakeChannelId}"
channelId / channelType	TangSeng 会话标识
chatType	direct 或 group
messageId / clientMsgNo / messageSeq	幂等和历史恢复所需标识
text	从 payload 中提取的文本，非文本可 fallback 为 URL/name
sender	{ id, name }
attachments	图片、音频、文件或 unsupported
mentions	TangSeng mention uids
directCatId / targetCatIds	前端显式路由目标

字段	含义
promptContext	Clowder 直聊的近期用户文本上下文
sourceRoleSnapshot	群角色快照

externalChatId 规则：

- 单聊：channelType = 1。虚拟 Clowder 直聊使用 TangSeng fake channel，避免不同用户共享同一个私聊频道。
- 群聊：channelType = 2，channelId 为群号。
- Clowder binding：connectorId + externalChatId -> threadId。

6.2 OutboundPayload

Clowder 通过 ImWebAdapter 回调 TangSeng：

字段	含义
connectorId	im-web
externalChatId	要写回的 IM 会话
threadId / invocationId	追踪 Clowder thread 和本次调用
catId / catDisplayName	回复来源 Agent
content / format	文本内容，默认 Markdown
richBlocks	Clowder rich blocks，前端可逐步映射
media	image、file、audio
reaction	silent reaction，附着到目标消息
stream	placeholder、chunk、final、cleanup
metadata	例如 replyToSender
platformMessageId	流式消息稳定 ID

TangSeng 出站转换规则：

- Markdown 文本落成普通文本消息，payload 带 markdown: true、ai: true。
- 图片和文件映射到 TangSeng content type。
- reaction 映射为 type = 1000 的 clowder_reaction，NoPersist = 1，RedDot = 0。
- cleanup 且内容为空时返回 no-op，不生成空消息。

6.3 UI 消息对象

前端消息对象由 normalizeMessage()、defaultMsg() 和 message-state.js 收敛，主要字段如下：

字段	含义
id 、 messageId 、 clientMsgNo 、 streamKey	去重和流式合并
senderId 、 senderName 、 senderAvatar	展示发送方
type	text、image、file、voice、system、deployment_card、project_group_confirmation 等
content	展示正文
status	sending、success、failed、revoked、edited
streaming	是否仍处于流式更新中
renderMode	Markdown 等渲染模式
replyRef	引用回复
mentions	@ 信息
reactions	普通 reaction 和本地 pending feedback
source	clowder 、 mock 等

6.4 ProjectGroupBinding

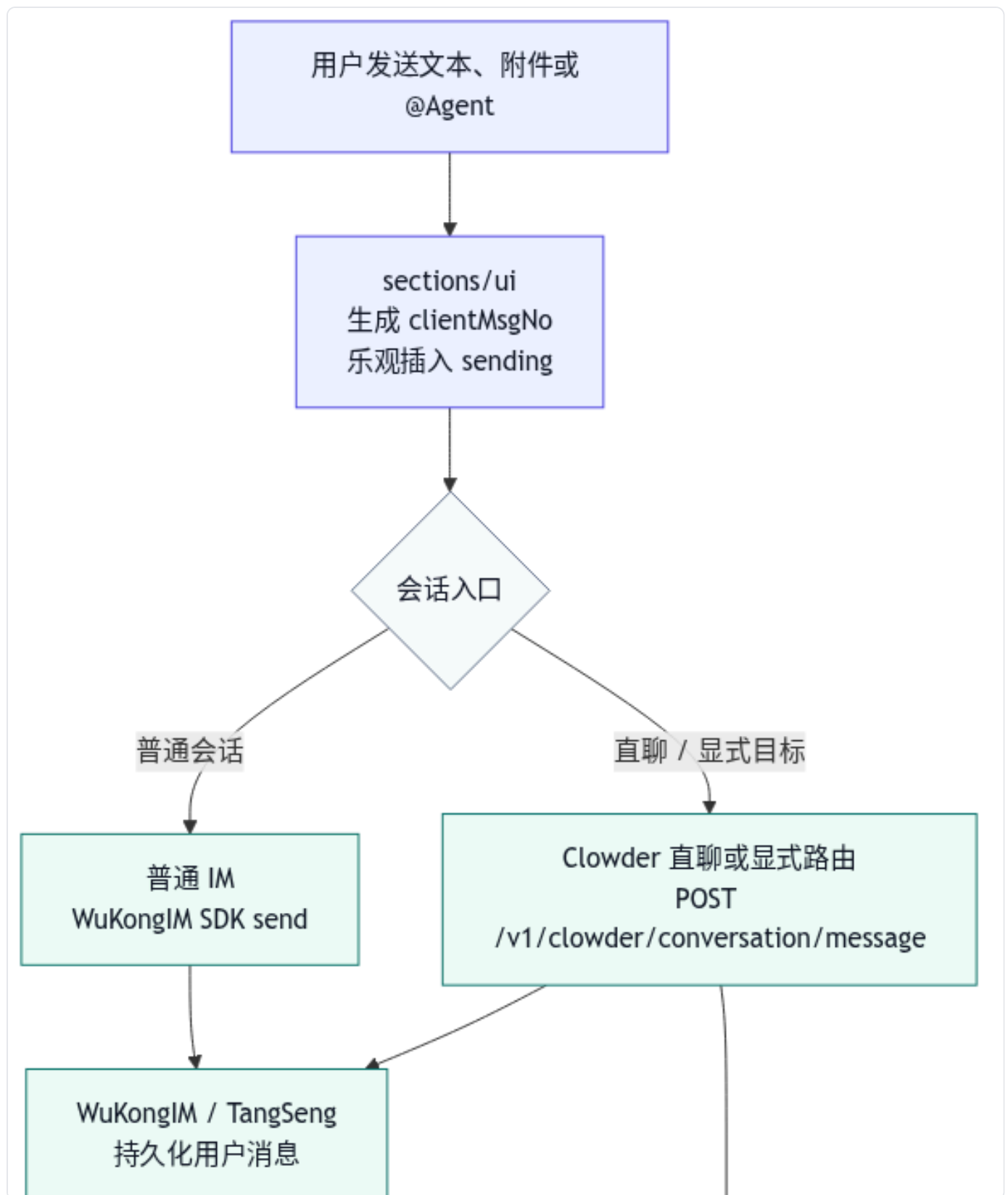
项目群 binding 记录 PM 直聊和项目群之间的关系：

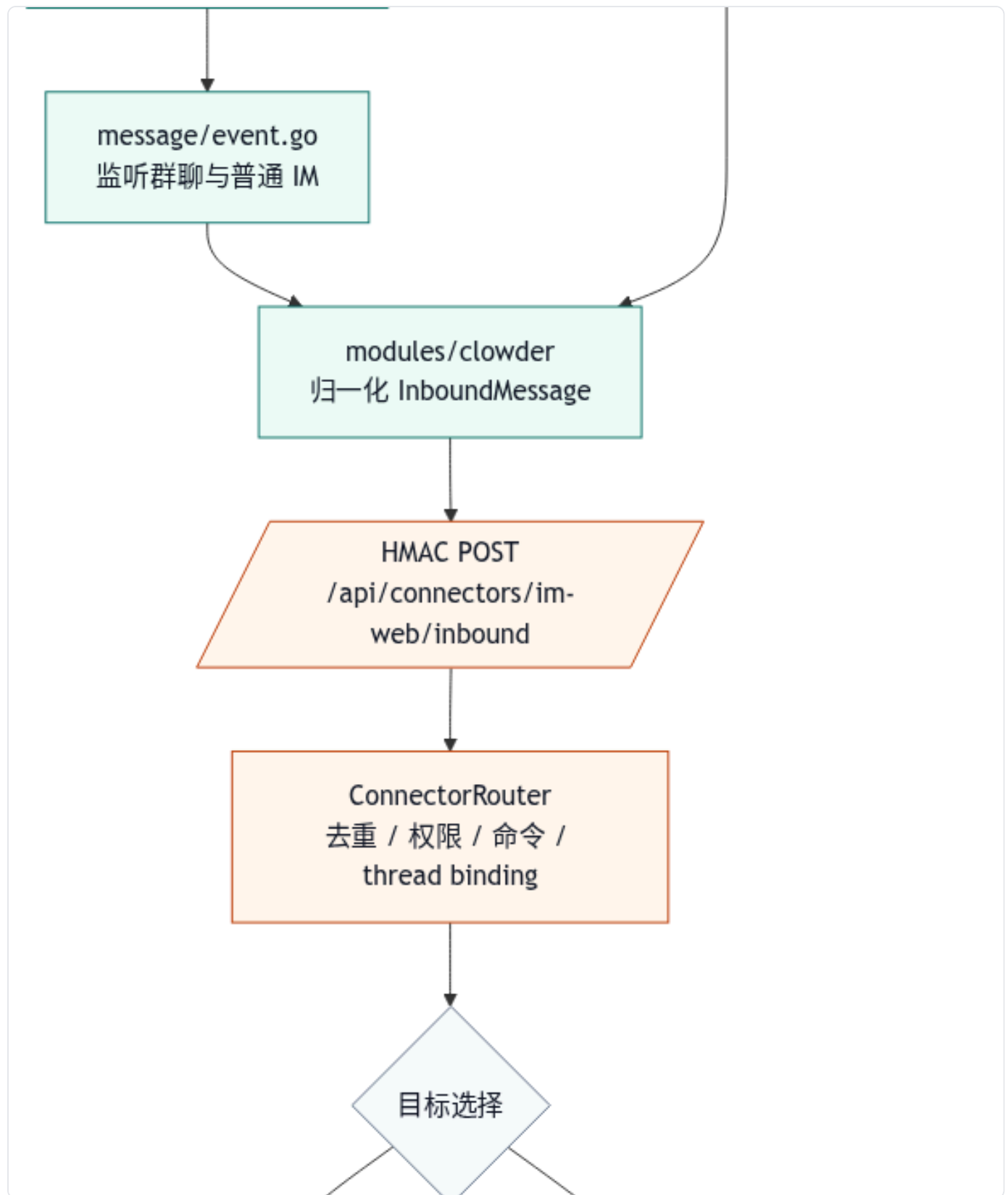
字段	含义
id	binding ID
userId	所属用户
projectName	规范化项目名
pmDirectChannelId / pmDirectChannelType	PM 直聊频道
pmDirectThreadId	PM 直聊 Clowder thread
projectGroupNo	TangSeng 群号
projectThreadId	项目群 Clowder thread
pmMemberId	PM 虚拟成员
userMemberIds / catMemberIds	成员集合
status	active 等

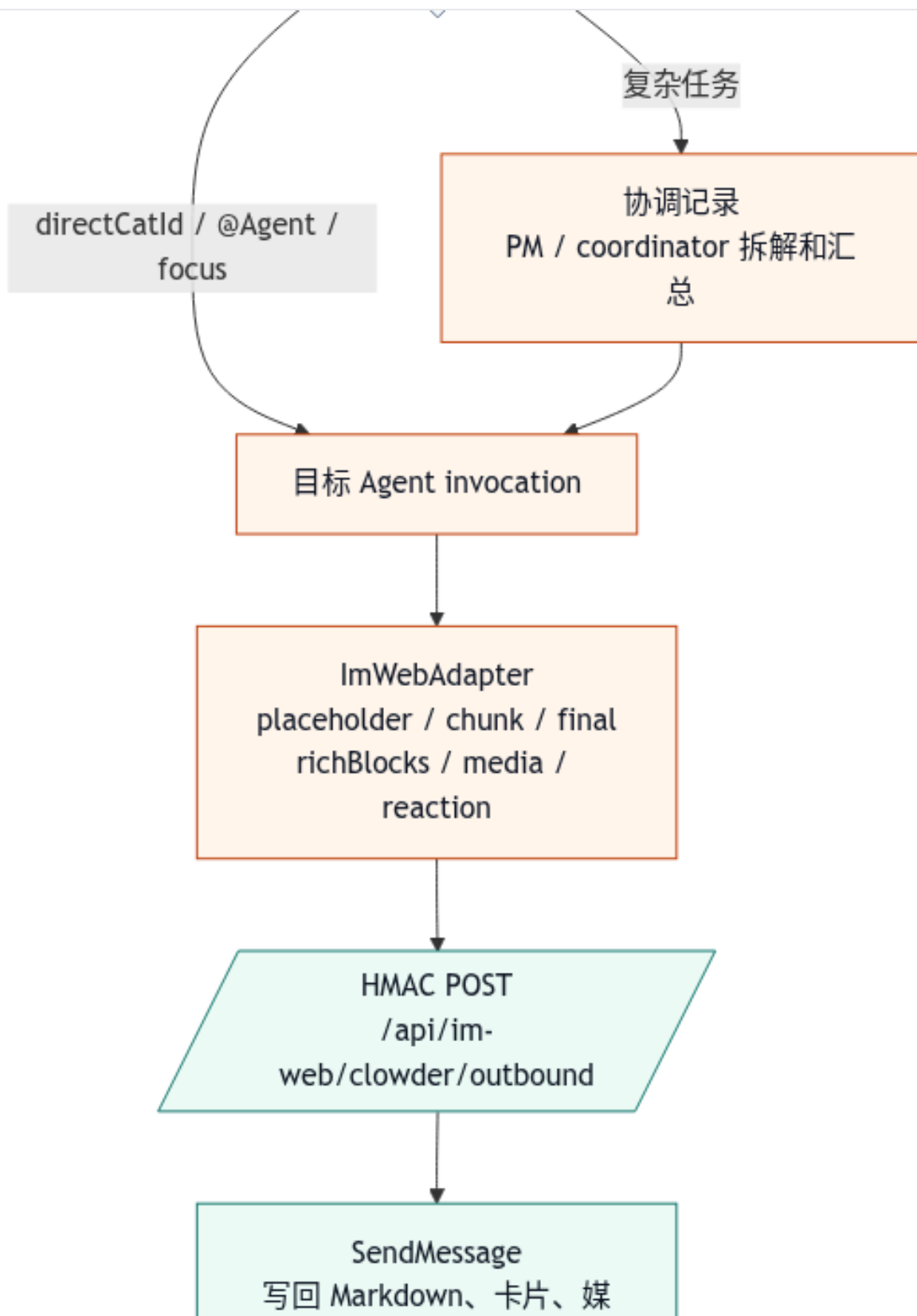
7. 消息生命周期

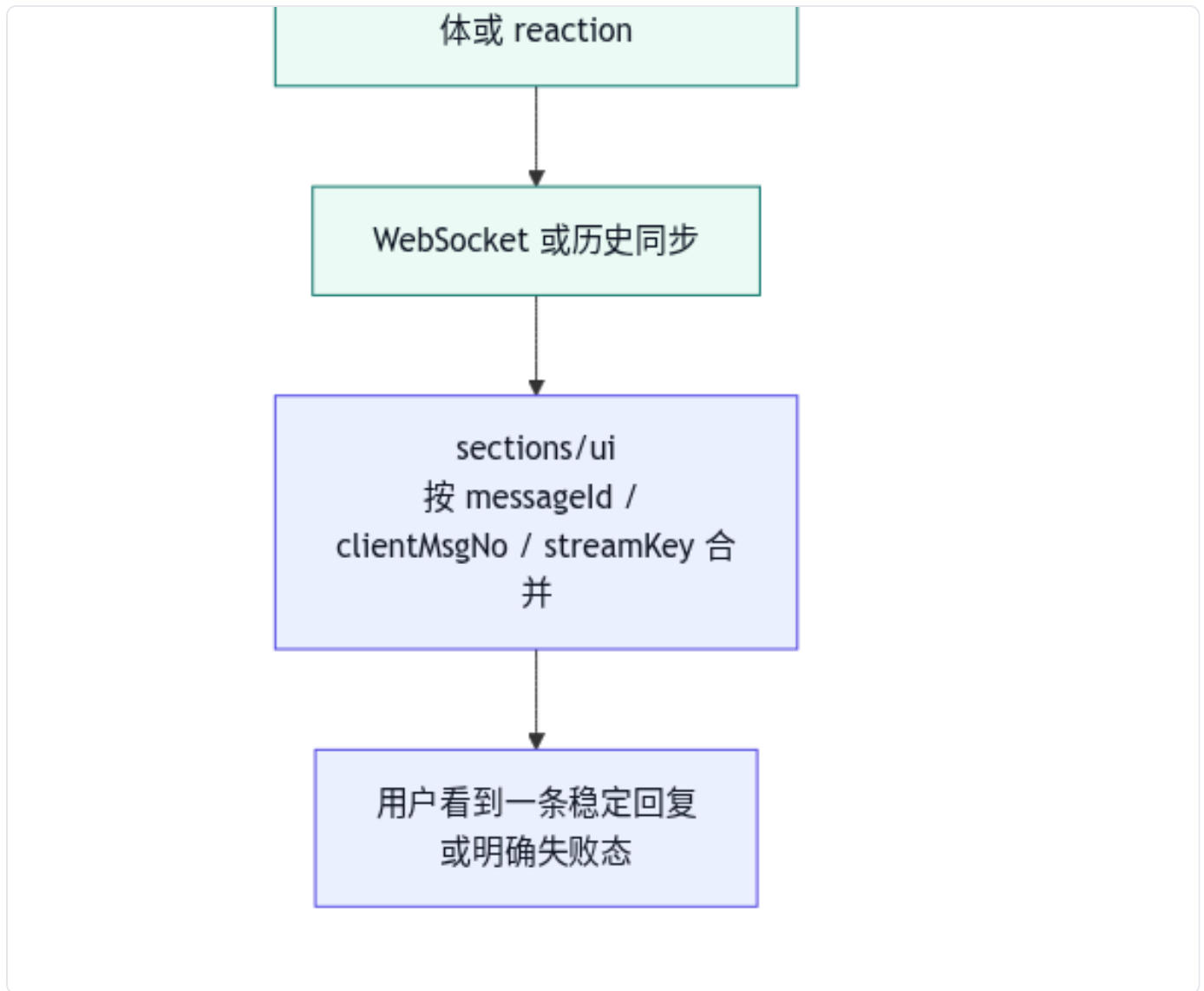
图源文件（.mmd 为 Mermaid 源文件，.md 为渲染中间格式，.png 为导出静态图）：

- `./figures/im-clowder-message-lifecycle.mmd`
- `./figures/im-clowder-message-lifecycle.md`
- `./figures/im-clowder-message-lifecycle.png`









7.1 普通 IM 消息

UI 调用 WuKongIM SDK `send()` → 本地插入 `sending` → SDK/历史同步返回后按 `clientMsgNo / messageId` 合并 → `conversationSummaryForMessage()` 更新会话摘要。

7.2 Clowder 直聊

UI 识别 `clowder_cat`: 会话 → 调 `/v1/clowder/conversation/message` (带 `directCatId` + `promptContext`) → TangSeng 先持久化用户消息 (`BuildInboundPersistMessage()`) 再转发 Clowder (`sendInboundTextWithRouting()`) → UI 对源消息加 pending feedback, Agent 回复到后清理。

7.3 群聊路由

`MessagesListenWithRoles()` 监听 → 查群成员角色 (`owner/manager/member/departed`) → `NormalizeInboundMessage()` 归一化 → Clowder `ConnectorRouter` 检查白名单、命令、@ → 明确目标优先, 复杂任务进协调者。

7.4 流式回复

流式回复用 `platformMessageId` 做稳定 ID，TangSeng 侧 `StableStreamClientMsgNo(invocationId, platformMessageId)` 生成 `client_msg_no`。UI 按 `streamKey` 合并 chunk，final 后状态变 `success`。cleanup 为空时返回 no-op。

8. 接口清单

8.1 UI 到 TangSeng

接口	方法	用途
<code>/v1/user/login</code>	POST	登录
<code>/v1/conversation/sync</code>	POST	同步会话
<code>/v1/message/channel/sync</code>	POST	同步消息历史
<code>/v1/file/upload</code>	GET	获取上传地址
<code>/v1/clowder/status</code>	GET	Clowder bridge 状态
<code>/v1/clowder/conversation</code>	GET	当前会话 Clowder 状态
<code>/v1/clowder/conversation/agents</code>	GET	会话可用 Agent
<code>/v1/clowder/cats</code>	GET/POST/DELETE	Agent 目录、创建、删除
<code>/v1/clowder/cats/connect</code>	POST	连接已有 Agent
<code>/v1/clowder/conversation/bind</code>	POST	绑定 IM 会话和 Clowder thread
<code>/v1/clowder/conversation/focus</code>	POST	设置 focus Agent
<code>/v1/clowder/conversation/focus/clear</code>	POST	清理 focus
<code>/v1/clowder/conversation/message</code>	POST	Clowder 直聊或显式路由消息
<code>/v1/clowder/group/cats/sync</code>	POST	同步群聊 Agent 列表
<code>/v1/clowder/group/cats</code>	GET	获取群聊 Agent
<code>/v1/clowder/project-groups/ensure</code>	POST	创建或复用项目群
<code>/v1/clowder/project-groups/active</code>	GET	查询活跃项目群 binding
<code>/v1/clowder/project-groups/:bindingId/thread</code>	POST	回写项目 thread
<code>/v1/clowder/skills/*</code>	GET/POST/PATCH/PUT/DELETE	技能市场、用户技能、分配、上传
<code>/v1/clowder/thread/:threadId/tasks</code>	GET	查询 thread tasks
<code>/v1/clowder/thread/:threadId/artifacts</code>	GET/POST	产物列表和登记
<code>/v1/clowder/thread/:threadId/manual-context-pins</code>	GET/POST/DELETE	长期上下文 pin

接口	方法	用途
/v1/clowder/conversation/deployment-request	POST/GET/PATCH	部署请求
/v1/clowder/conversation/deployment-action	POST	部署确认、取消、重试

8.2 TangSeng 到 Clowder

Clowder 接口	方法	用途
/api/connectors/im-web/inbound	POST	IM 消息进入 ConnectorRouter
/api/connectors/im-web/agents	GET	获取指定 externalChatId 可用 Agent
/api/cat-templates	GET	获取角色模板和默认模型
/api/connectors/im-web/deployment-requests	POST/GET/PATCH	代理部署请求
/api/connectors/im-web/deployment-action	POST	代理部署动作
/api/threads/:threadId/tasks	GET	任务
/api/threads/:threadId/artifacts	GET/POST	产物
/api/threads/:threadId/manual-context-pins	GET/POST/DELETE/PATCH	长期上下文

8.3 Clowder 到 TangSeng

TangSeng 接口	方法	用途
/api/im-web/clowder/outbound	POST	Clowder 出站消息、流式更新、媒体、reaction 回写 IM

9. 鉴权和安全

9.1 双向 HMAC

TangSeng 转发 inbound 到 Clowder:

- Header: x-im-web-timestamp 、 x-im-web-signature
- 签名: HMAC_SHA256(secret, timestamp + "." + rawBody)
- Clowder 默认容忍 5 分钟时间差。

Clowder 回调 outbound 到 TangSeng:

- Header: x-clowder-timestamp 、 x-clowder-signature
- TangSeng 使用同一 secret 校验。
- 签名缺失、时间过期、签名不匹配都返回 401。

9.2 用户身份

- 浏览器访问 TangSeng 仍使用 TangSeng token。

- TangSeng 代理访问 Clowder 时使用 `CLOWDER_DEFAULT_OWNER_USER_ID` 写入 `x-cat-cafe-user`，避免把 Clowder 管理权限暴露给浏览器。
- 群消息会附带 `sourceRoleSnapshot`，用于 Clowder 判断管理员命令和群授权。

9.3 文件和技能安全

- 聊天文件先通过 TangSeng 文件上传接口拿上传地址，再发送媒体消息。
- Clowder 技能上传拒绝不安全 zip 路径。
- 富媒体和 rich blocks 先走白名单式展示；未支持类型降级为文本或通用文件。

10. 幂等、恢复和失败处理

风险	处理
重复 webhook	<code>InboundMessageDedup</code> 按 connector、externalChatId、externalMessageId 跳过
本地消息和服务端回包重复	UI 按 <code>messageId</code> 、 <code>clientMsgNo</code> 合并
流式 chunk 重复或乱序	UI 按 <code>streamKey</code> 合并，final 覆盖最终内容
cleanup 产生空消息	TangSeng 返回 <code>ErrOutboundNoop</code>
群虚拟 Agent 发送者不可投递	TangSeng 确保虚拟用户存在，群聊必要时改用真实订阅者发送
Clowder 不可达	API 返回明确错误码，UI 保留失败态
SDK 缺失或不可用	mock 会话可本地成功，真实会话给失败或同步错误
项目群重复创建	project group key 由 user、PM direct channel、项目名组成，active binding 优先复用

11. 前端实现要点

11.1 消息发送

`message.js` 中 `sendNativeMessage()` 是主要入口：

1. 根据当前会话生成 identity。
2. 本地插入消息。
3. 文本走普通 SDK 或 Clowder conversation message。
4. 媒体先上传再发送。
5. 成功后缓存 prompt context。
6. 失败时更新 `nativeError` 和 `syncError`。

11.2 消息合并

`message-state.js` 处理这些场景：

- `mergeNativeMessageIntoList()` 合并服务端消息。

- `mergeAgentReplyEventIntoList()` 合并 Agent placeholder/chunk/final。
- `applyReactionEventIntoList()` 把 silent reaction 加到源消息上。
- `applyAgentPendingFeedbackIntoList()` 在本地给“正在处理”反馈。
- `mergeSyncedMessagesPreservingLocalContext()` 同步历史时保留本地上下文消息。

11.3 Agent 目录

Agent 目录要把多种来源整理成同一个 UI 结构：

- 静态系统 Agent。
- Clowder `/v1/clowder/cats` 返回的 Agent。
- Clowder direct fallback `/api/connectors/im-web/agents`。
- `cat-template.json` 中的模板。
- 用户创建的运行时 Agent。

直聊联系人 ID 用 `clowder_cat:` 前缀，但展示名优先来自 Clowder directory 和模板，避免 UI 暴露技术 ID。

11.4 卡片

当前重点卡片：

卡片	文件	状态
部署确认卡	<code>services/native-im/deployment.js</code> 、 <code>DeploymentCard.vue</code>	submitting、queued、running、succeeded、failed、cancelled
项目群确认卡	<code>services/native-im/project-group.js</code> 、 <code>CoordinatorProjectGroupCard.vue</code>	pending、creating、created、failed、cancelled
模板 Agent 确认卡	<code>services/native-im/coordinator-template-cats.js</code> 、 <code>CoordinatorTemplateCatsCard.vue</code>	pending、creating、created、partial、failed、cancelled
文件卡	<code>FileCard.vue</code> 、 <code>FilePreviewPanel.vue</code>	下载、预览、失败

12. 后端实现要点

12.1 Inbound

`registerClowderBridgeListener()` 在配置完整时注册监听器。每次收到 IM 消息：

1. 群聊查询成员角色。
2. 生成 `RawIMMessage`。
3. `NormalizeInboundMessage()` 提取文本、mention、attachment。
4. `Client.ForwardInbound()` 通过 HMAC 发送到 Clowder。

12.2 Outbound

`/api/im-web/clowder/outbound` 的处理流程：

1. 读 raw body。
2. 校验 `x-clowder-*` 签名。
3. 解析 `OutboundPayload`。
4. 构建 TangSeng `MsgSendReq`。
5. 确保虚拟 Clowder 用户存在。
6. 群聊中加载订阅者，必要时改写发送者。
7. 调用 `ctx.SendMessage()` 写回 IM。

12.3 Proxy

TangSeng bridge 代理 Clowder 的部分能力，目的是保持浏览器只访问 TangSeng：

- Agent directory。
- Cat templates。
- Skills。
- Project group。
- Deployment requests。
- Manual context pins。
- Thread tasks/artifacts/workspaces。

13. Clowder 路由要点

`ConnectorRouter.route()` 的顺序需要保持稳定：

1. 去重。
2. 发送 instant ACK reaction。
3. 群白名单检查。
4. slash command 拦截。
5. 媒体下载与语音转写。
6. 查找或创建 thread binding。
7. 写入 connector message。
8. 解析 @ 和显式目标。
9. 必要时创建 coordination record。
10. 触发 Agent invocation。

`im-web` 的目标选择规则：

- `directCatId` / `targetCatIds` 优先。
- 文本 @ 命中优先。
- 无明确目标时，优先协调者。
- 已有活跃参与者可作为 fallback。

14. Pin 与上下文

这里有两个容易混淆的概念：

名称	所属层	用途
IM Pin	TangSeng message module	聊天窗口置顶消息，多端同步，受群主/管理员/群配置限制
Manual context pin	Clowder thread	给 Agent prompt 的长期上下文，可标记源消息状态

产品上可以互相引用源消息，但不要把权限、数据表 and 状态混成一套。

15. 测试与验证

15.1 已有测试入口

类型	路径
TangSeng bridge normalization/outbound/listener	sections/im/TangSengDaoDaoServer/modules/clowder/*_test.go
TangSeng bridge config/signature	sections/im/TangSengDaoDaoServer/modules/common/clowder*_test.go
Clowder connector/router/adapter	sections/clowder-ai/packages/api/src/infrastructure/connectors/* 对应测试
UI native IM 状态	sections/ui/tests/native-im.test.mjs
UI 视觉截图	assets/roadmap/evidence/
UI 效果证据	assets/roadmap/evidence/ 中的群聊、AI 回复、文件预览、智能体看板和文件卡截图

15.2 建议门禁

```
cd sections/ui && npm test
cd sections/ui && npm run build
cd sections/im/TangSengDaoDaoServer && go test ./modules/common ./modules/clowder
cd sections/clowder-ai && pnpm test
cd sections/clowder-ai && pnpm build
```

Mermaid 图验证：

```
npx -y @mermaid-js/mermaid-cli@latest -i assets/roadmap/figures/agenthub-im-clowder-architecture.mmd -o /tmp/agenthub-architecture.png
npx -y @mermaid-js/mermaid-cli@latest -i assets/roadmap/figures/im-clowder-message-lifecycle.mmd -o /tmp/im-clowder-message-lifecycle.png
```

15.3 手工 smoke

流程	检查点
普通聊天	发送、失败、重试、刷新历史
Clowder 直聊	directCatId、promptContext、ACK、流式合并
群聊 @	群角色快照、白名单、目标 Agent、回写群聊
项目群	PM 直聊创建项目群、复用 binding、打开项目群、同步 Agent
部署卡	创建、确认、取消、重试、日志
技能	市场、添加、分配、删除、上传安全
移动端	375px 输入区、文件卡、长按菜单、右侧面板退路

16. 已知风险

优先级	风险	影响	建议
P0	项目群 binding 当前以内存态为主	重启后可能需要重新发现或补持久化	将 active binding 落库，并补迁移测试
P1	ClowderPanel.vue 仍有部分模拟任务数据	右侧面板可能与真实 thread 状态不一致	逐步接入 /v1/clowder/thread/* 和 project group active binding
P1	真实 Clowder 与 TangSeng 启动顺序不固定	首次进入可能显示 unconfigured 或 unavailable	UI 区分 disabled、unconfigured、unreachable
P2	rich blocks 专属组件不完整	一些 Agent 输出只能以 Markdown 或文本显示	先按使用频率补部署、artifact、diff、checklist
P2	移动端右侧工作区路径仍需打磨	面板可能占屏或返回路径不统一	用独立路由承接 Clowder panel、文件预览和部署日志

17. 维护建议

- **Clowder 能力扩展**：先决定新能力属于聊天消息、右侧面板、Agent 看板还是设置页，再开始实现。
- **卡片**：每增加一种卡片，同时定义消息类型、状态机、失败态、刷新恢复路径和截图验收点。
- **Connector 字段**：扩展时优先改 InboundMessage / OutboundPayload，不要在 UI 里解析临时字符串。
- **群聊权限**：新增群聊能力前，写清楚普通成员、管理员、群主和 departed 成员各自的行为。
- **角色模板**：每次更新后同步检查前端展示名、别名、能力标签和直聊联系人。